

1. Introduccion

Test Driven Development (TDD) es una tecnica donde las pruebas se escriben antes del codigo de produccion. Ciclo Red-Green-Refactor (Kent Beck, 'Test Driven Development: By Example', 2002): escribir prueba que falla (RED), codigo minimo para que pase (GREEN), refactorizar (REFACTOR).

'Growing Object-Oriented Software, Guided by Tests' (Freeman & Pryce) extiende TDD a nivel de sistemas: el software crece organicamente guiado por pruebas, en lugar de diseno upfront.

1.1 El Ciclo TDD

- RED: prueba que falla para funcionalidad aun no implementada.
- GREEN: codigo minimo para que la prueba pase (sin optimizar).
- REFACTOR: mejorar el codigo manteniendo todas las pruebas verdes.

1.2 Beneficios Medibles

- 40-80% menos bugs en produccion (estudios empiricos, Nagappan et al., 2008).
- Diseno mas modular y desacoplado (el testing force a interfaces limpias).
- Documentacion viva: las pruebas describen comportamiento esperado.
- Refactoring seguro: cualquier cambio verificado al instante.

2. TDD Aplicado al Proyecto

2.1 Testeo de knowledge.py (algoritmo de scoring)

El corazon del RAG es el scoring por terminos. TDD exige probar los casos limite:

```
class KnowledgeTest(unittest.TestCase):
    def test_retrieve_encuentra_coincidencia_parcial(self):
        resultado = retrieve_snippets("que es RAG")
        self.assertTrue(len(resultado) > 0)
        self.assertIn("rag", [r["id"] for r in resultado])

    def test_retrieve_sin_coincidencia_retorna_vacio(self):
        resultado = retrieve_snippets("zyxwvu teoria incognita")
        self.assertEqual(resultado, [])

    def test_retrieve_respeto_limite_de_resultados(self):
        resultado = retrieve_snippets("curso")
        self.assertLessEqual(len(resultado), 2)
```

2.2 Testeo de server.py (API REST)

```
class BaseApiTest(unittest.TestCase):
    # Ver ejemplos en BDD (seccion 2)
    # Los tests base ya cubren: health, ask exitoso, ask con error
```

2.3 Testeo de context_store.py (CAG)

Para implementar CAG, TDD dicta escribir estas pruebas primero:

```
class ContextStoreTest(unittest.TestCase):
    def test_save_persiste_clave_valor(self):
        store = ContextStore()
        store.save("ana", "style", "analogias")
        ctx = store.list_for_user("ana")
        self.assertIn({"key": "style", "value": "analogias"}, ctx)

    def test_contexto_aislado_por_usuario(self):
        store = ContextStore()
        store.save("ana", "x", "1")
        ctx_luis = store.list_for_user("luis")
        self.assertEqual(ctx_luis, [])

    def test_sobrescribe_misma_clave(self):
        store = ContextStore()
        store.save("ana", "tema", "SDD")
        store.save("ana", "tema", "BDD")
        ctx = store.list_for_user("ana")
        self.assertEqual(len(ctx), 1)
        self.assertEqual(ctx[0]["value"], "BDD")
```

3. Piramide de Testing

Proyecto sigue la piramide clasica:

- Base: tests unitarios (knowledge, context_store) - rapidos, aislados.
- Medio: tests de integracion (server con puerto real).

- Tope: tests de validacion (CAG contract) - cobertura de requisitos del examen.

4. Referencias

Test Driven Development: By Example (Kent Beck) - Addison-Wesley

Growing Object-Oriented Software, Guided by Tests (Freeman & Pryce) - Addison-Wesley

xUnit Test Patterns (Gerard Meszaros) - Addison-Wesley

Pytest Documentation - [pytest.org](https://docs.pytest.org/)

Nagappan et al., 'Realizing quality improvement through test driven development' (2008)